Type-Safe Method Inlining

Jens Palsberg

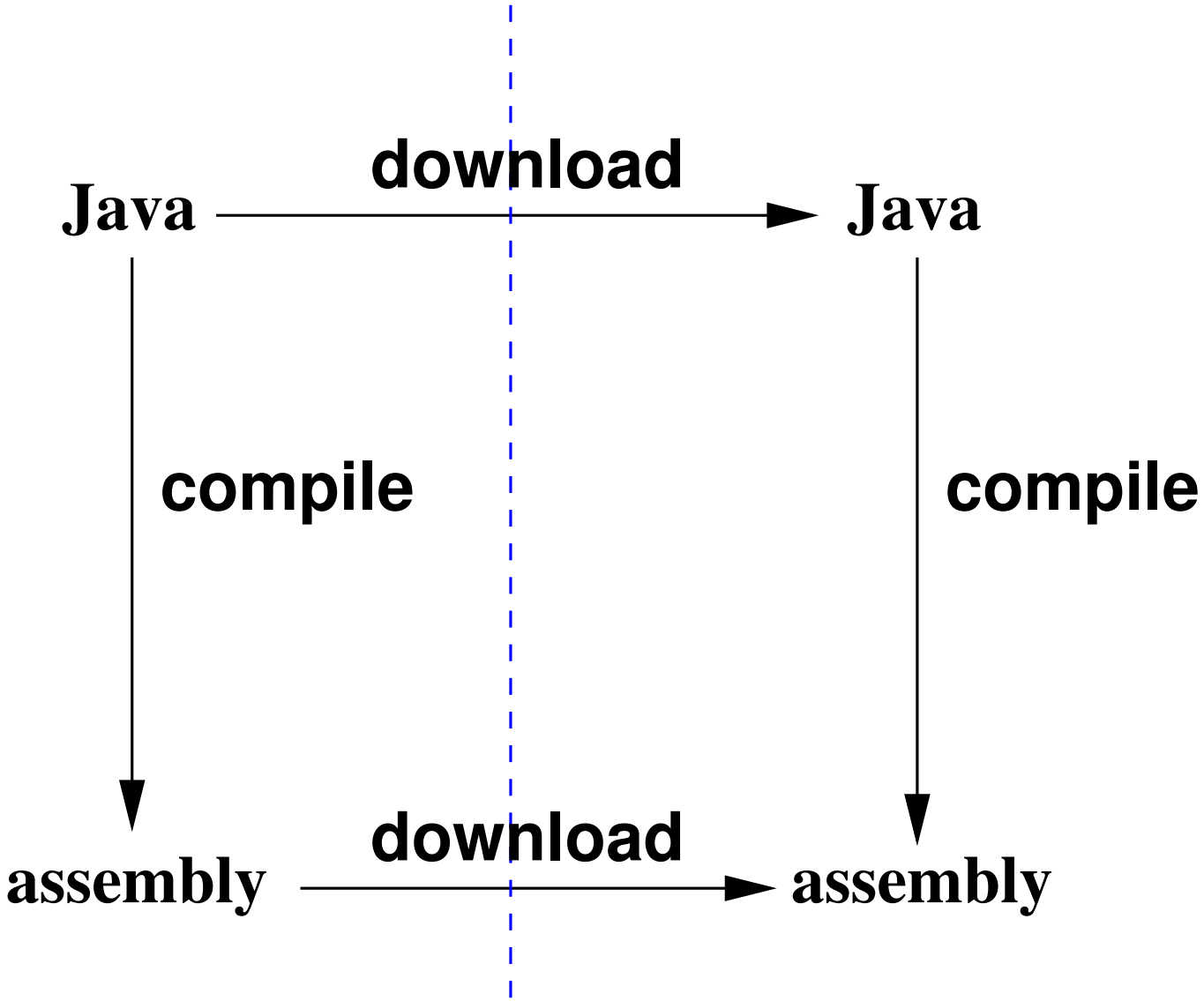
UCLA Computer Science Department

University of California, Los Angeles

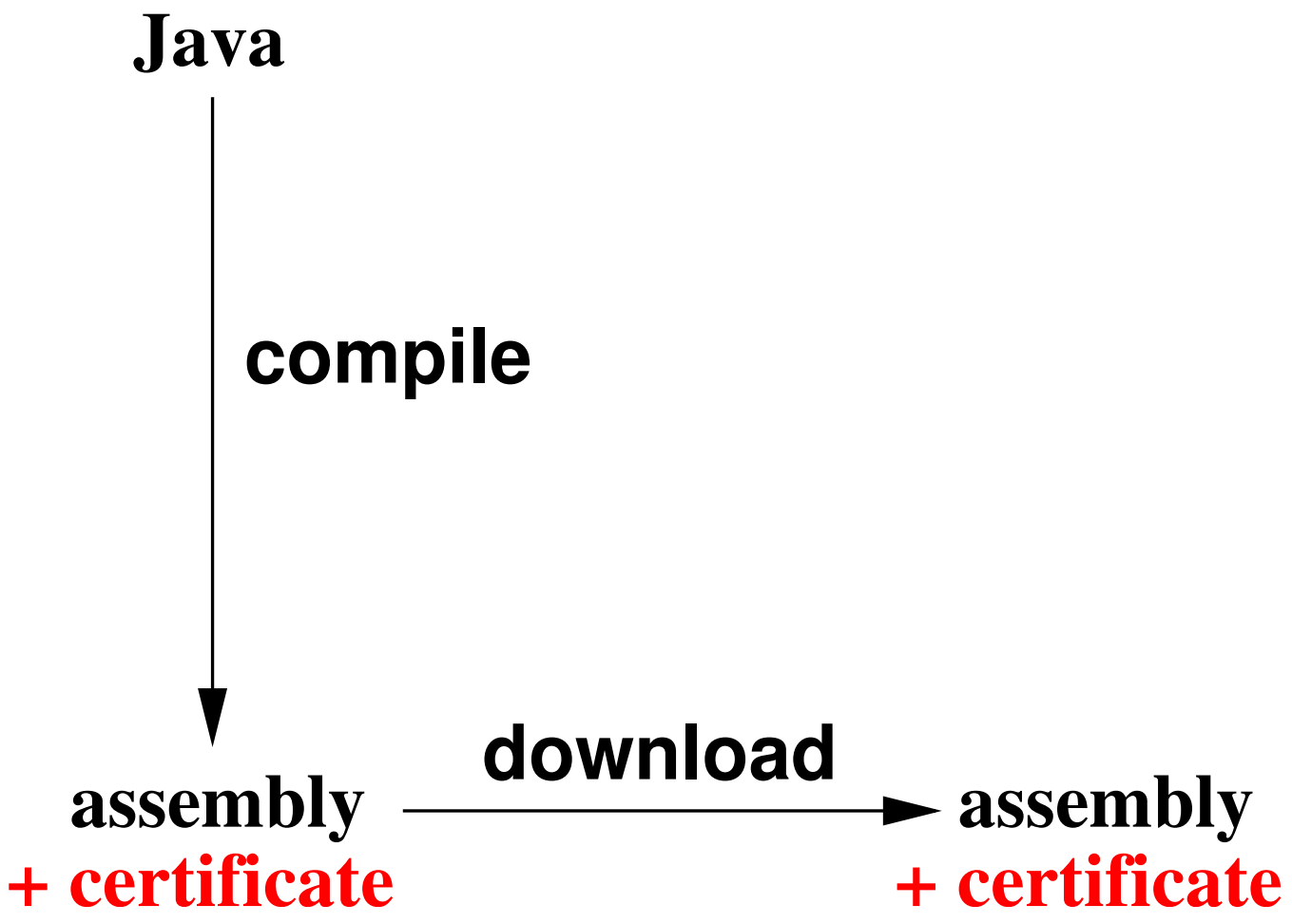
Supported by NSF and DARPA.

Joint work with Neal Glew [ECOOP 2002, FTJLP 2004]

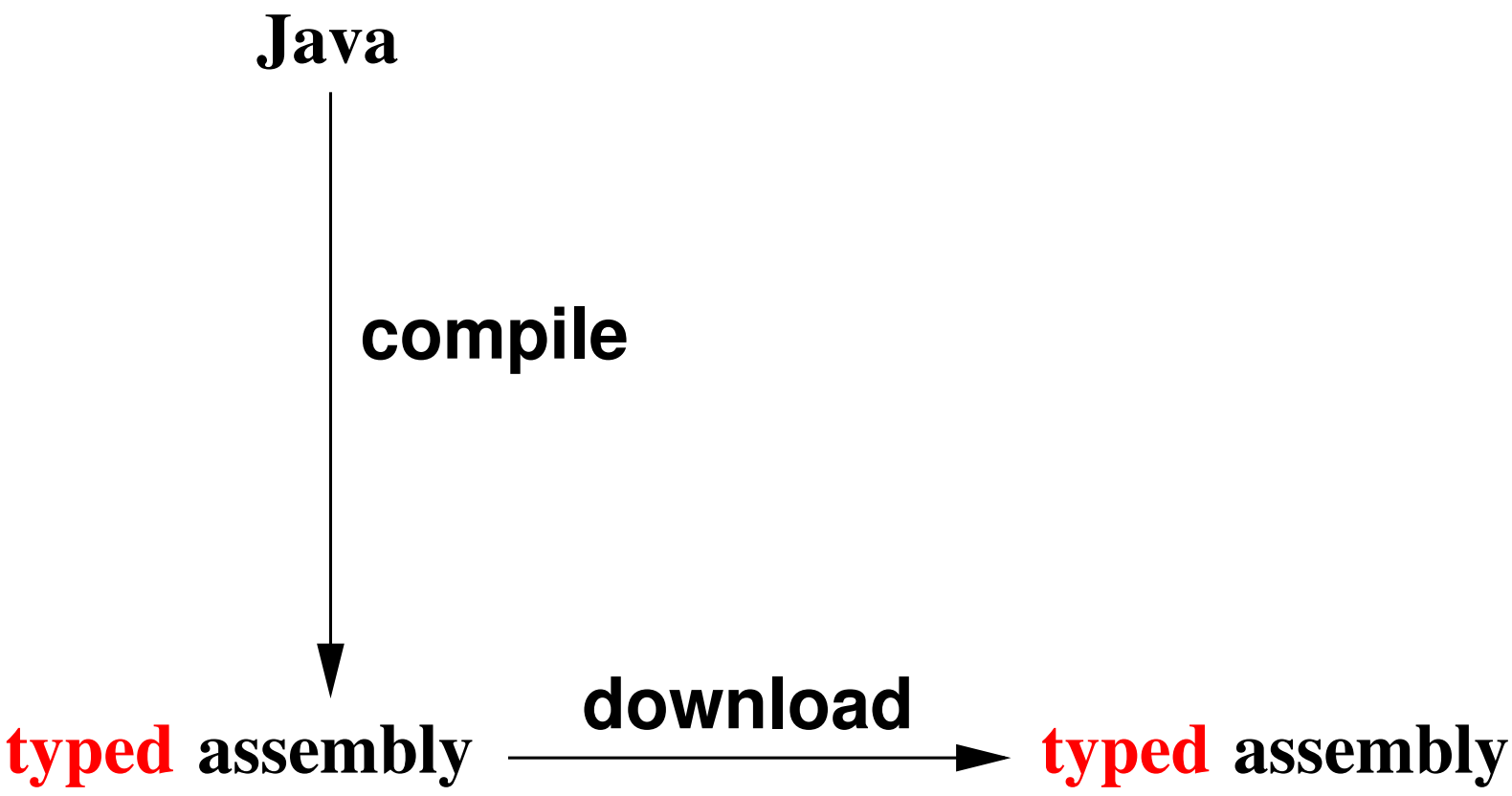
Compilers


Java
download
Java
compile
compile
assembly
download
assembly

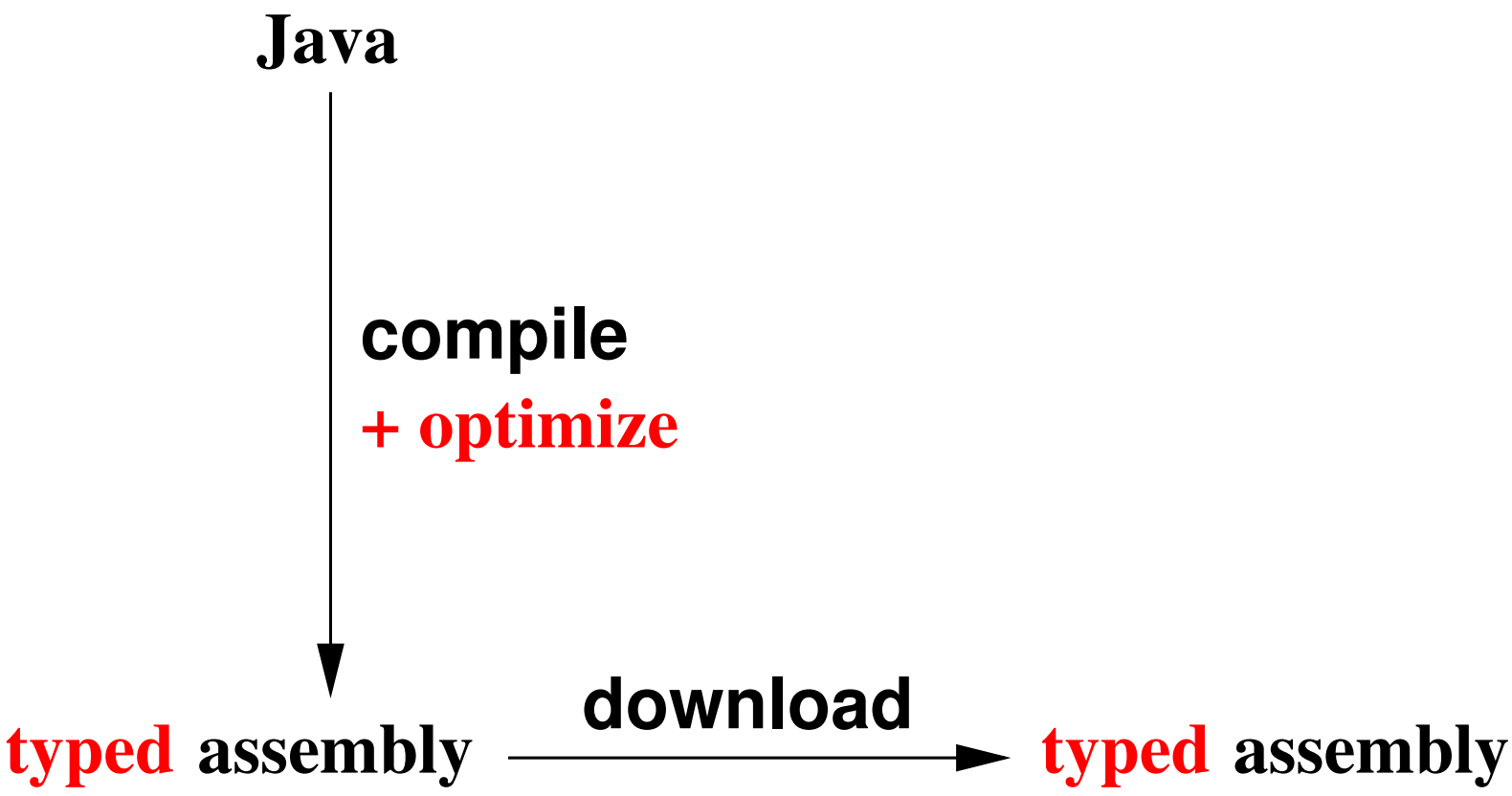
Certifying Compilers



Type-Preserving Compilers



Certifying Optimizing Compilers



Typed Intermediate Languages

[Modern Compiler Implementation in Java,
by Appel and Palsberg, 2002]
[Palsberg, tutorial at PLDI 2002]

Types help debugging the compiler.

Project in my undergraduate compiler course:

MiniJava
↓
Piglet
↓
Spiglet
↓
Kanga
↓
MIPS

Our Results

New approach to method inlining:

- New flow analysis: TSMI.
- Key idea 1: align the flow analysis with the Java type system.
- Key idea 2: change the type annotations.

Result: method inlining is type safe.

Theorems

Experiments

Conclusion: our approach is correct, efficient, and scalable, and inlines many method calls.

Example of Method Inlining (1/3)

Method inlining is used by all Java virtual machines,
by bytecode compaction tools such as JAX, etc.

```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

Example of Method Inlining (2/3)

```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

Example of Method Inlining (3/3)

```
A x = new A();
B y = new B();

new Q().p();
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

Benchmark Characteristics

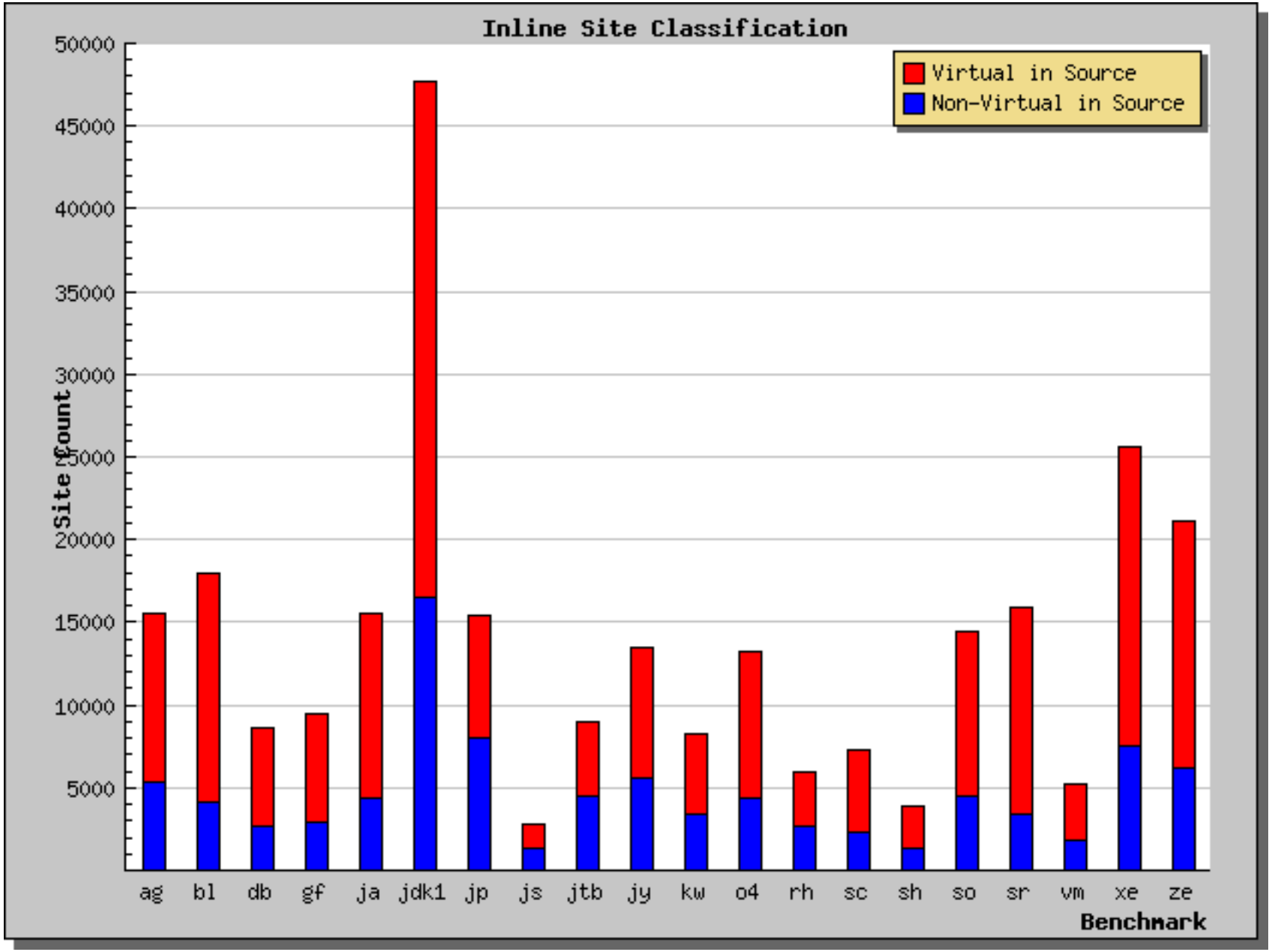
The Purdue Java benchmark suite:

100,000	class files corresponding to
10,000,000	lines of Java source code.

In this talk, I will concentrate on:

27,000	class files corresponding to
2,300,000	lines of Java source code.

How many calls can be inlined?



Enabling technology: flow analysis

Goal: find call sites with unique callee.

Set-based analysis: the flow set for an expression is a set of class names.

Idea: flow set for e = { A, B, C } means
e will evaluate to either an A-object, a B-object, or a C-object

CHA → **TSMI** → **0–CFA**

→ **cost and accuracy**

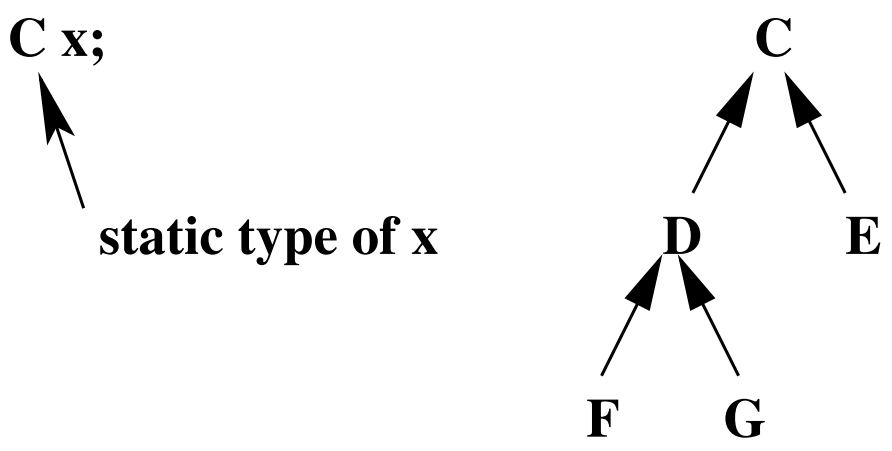
Class Hierarchy Analysis (CHA)

CHA relies on the type system; it is a type-based analysis.

[Dean, Grove, Chambers ECOOP 1995]

Flow set = all subtypes of the static type

class hierarchy (excerpt)



Flow set for x = { C, D, E, F, G }

Example of Class Hierarchy Analysis (1/2)

```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

Example of Class Hierarchy Analysis (2/2)

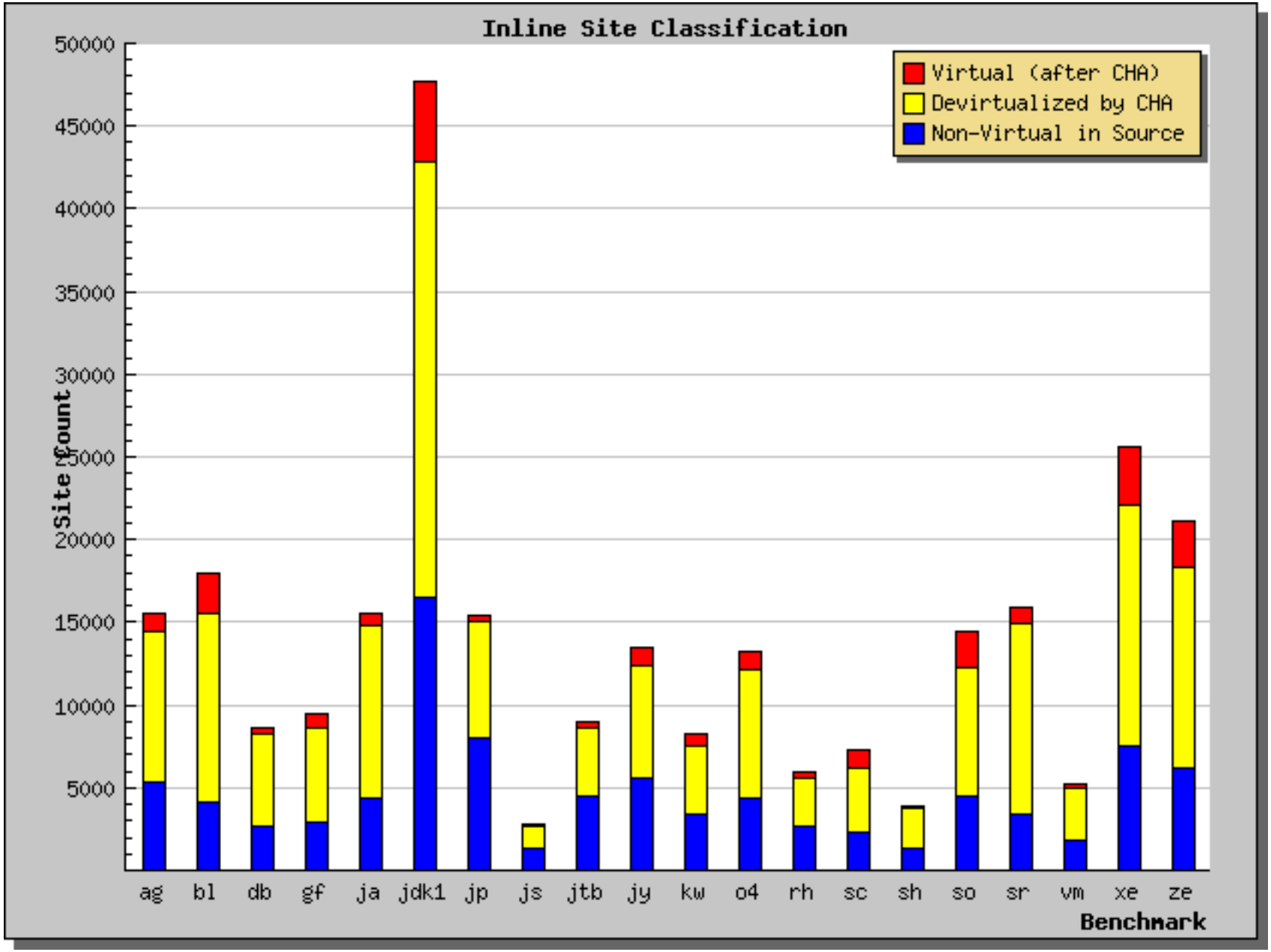
```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

How good is CHA?



0-CFA

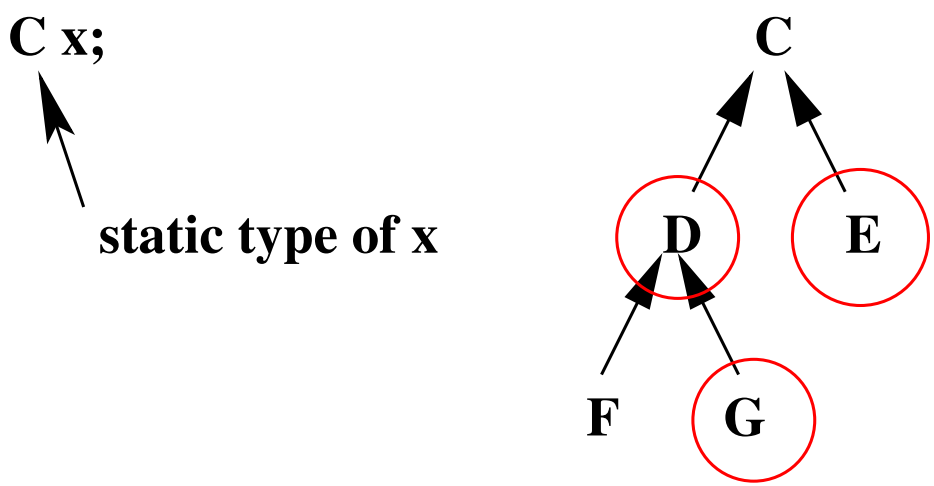
Flow insensitive, context insensitive flow analysis.

[Palsberg et al. OOPSLA 1991, ECOOP 1992]

Called 0-CFA by Shivers in 1992. Can be done in $O(n^3)$ time.

Note: 0-CFA does not rely on the type system, but is type respecting.

`class hierarchy (excerpt)`



`Flow set for x = { D, E, G }`

0-CFA Constraints

Approach: (1) generate constraints, (2) solve constraints.

For each expression e, there is a flow variable [e].

Program	Constraints
new C()	$C \in [\text{new } C()]$
x = e;	$[x] \supseteq [e]$
$e_1.m(e_2)$ and	$C \in [e_1] \Rightarrow [e_2] \subseteq [a]$ $C \in [e_1] \Rightarrow [e] \subseteq [e_1.m(e_2)]$

```
class C {
    ...
    B m(A a) {
        ...
        return e;
    }
}
```

Example of 0-CFA (1/2)

```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

Example of 0-CFA (2/2)

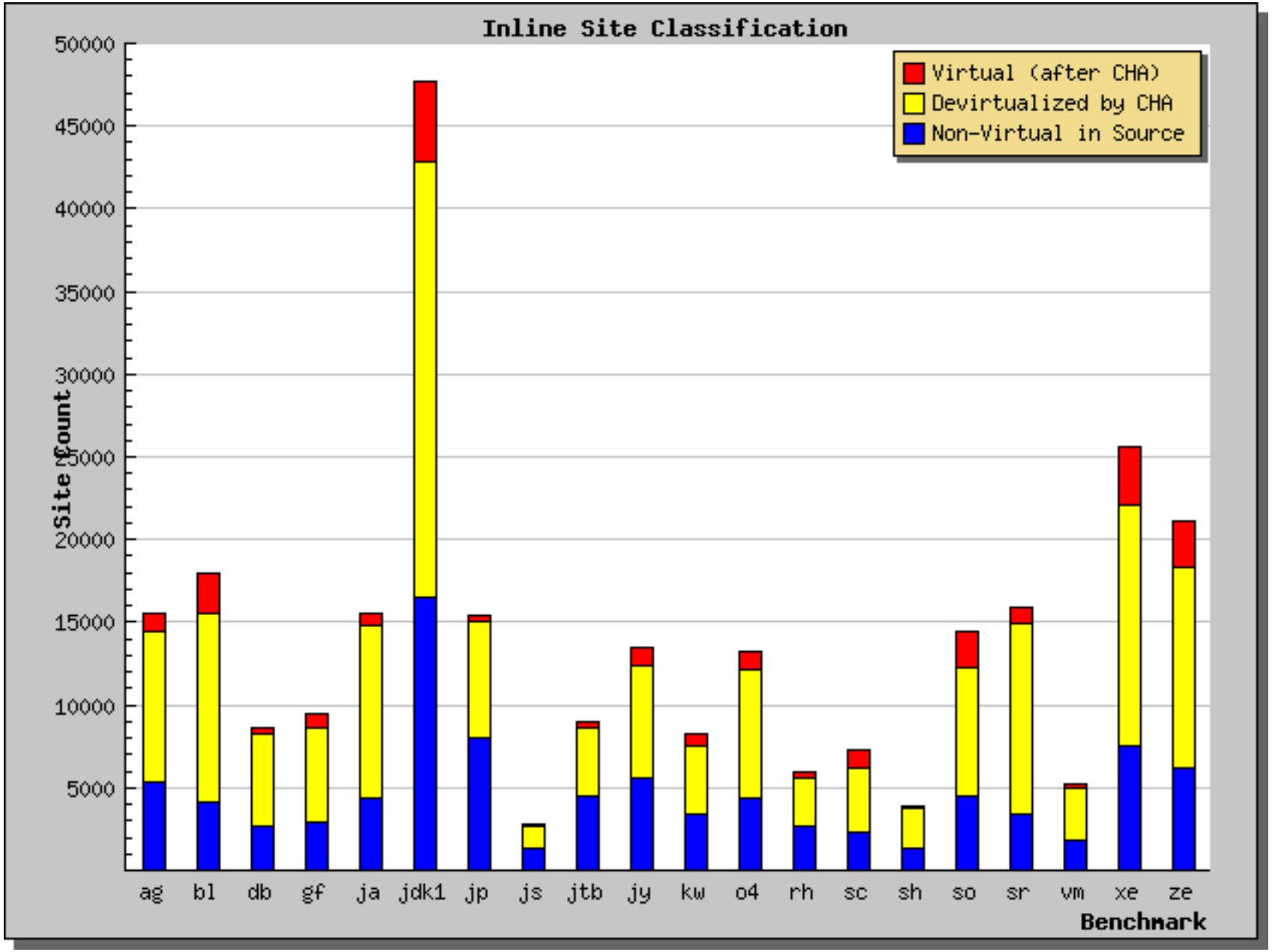
```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

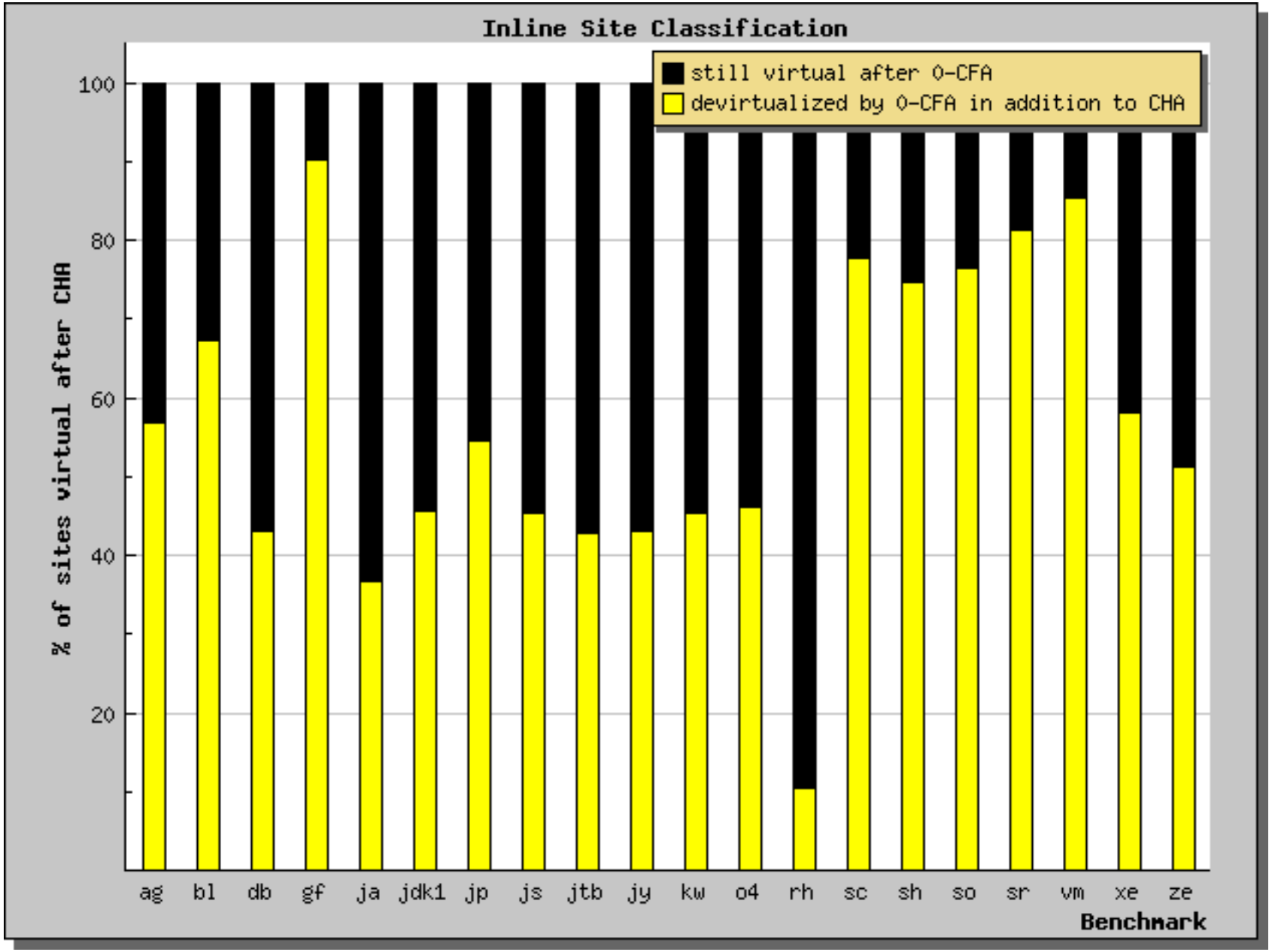
```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

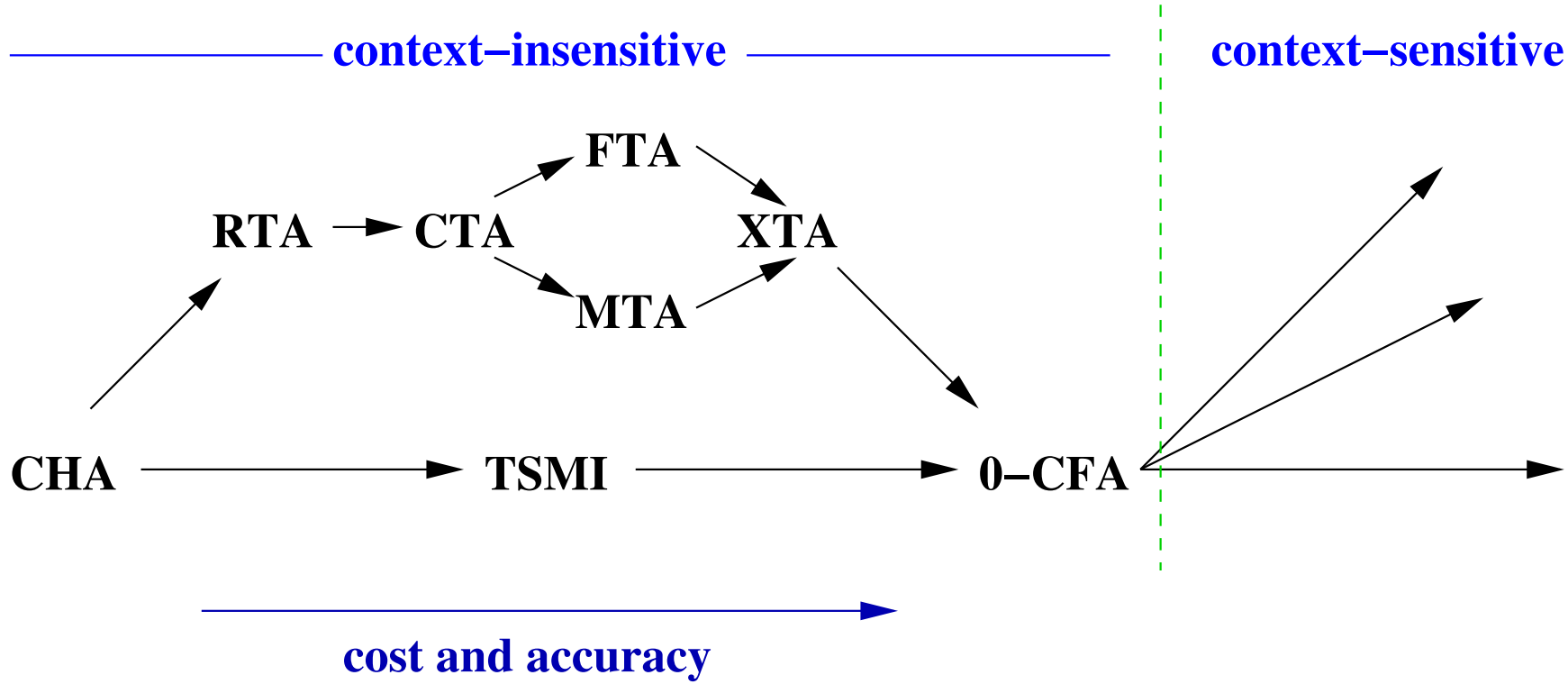
How good is CHA? (revisited)



How good is 0-CFA?



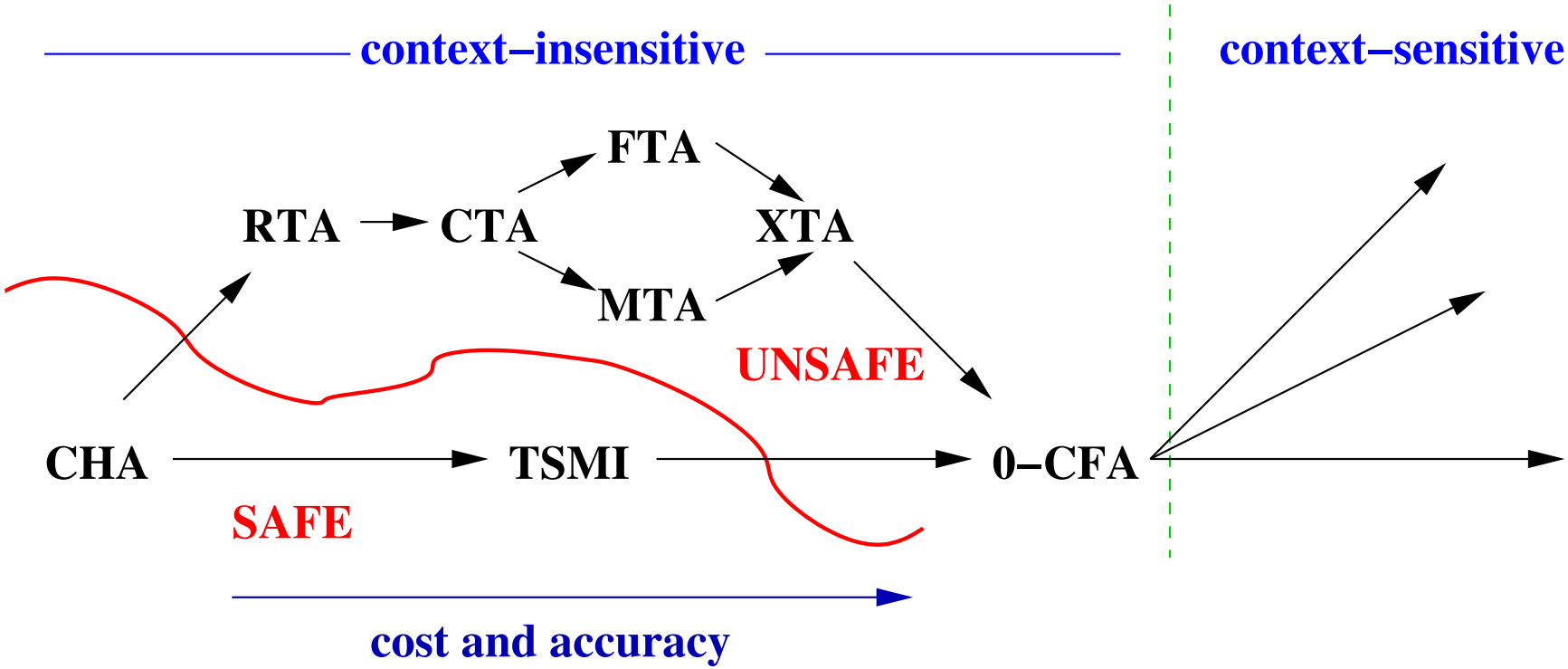
Is there a middle ground?



[Tip and Palsberg, OOPSLA 2000]

See also [Hendren et al, OOPSLA 2000]

Optimization versus Typability

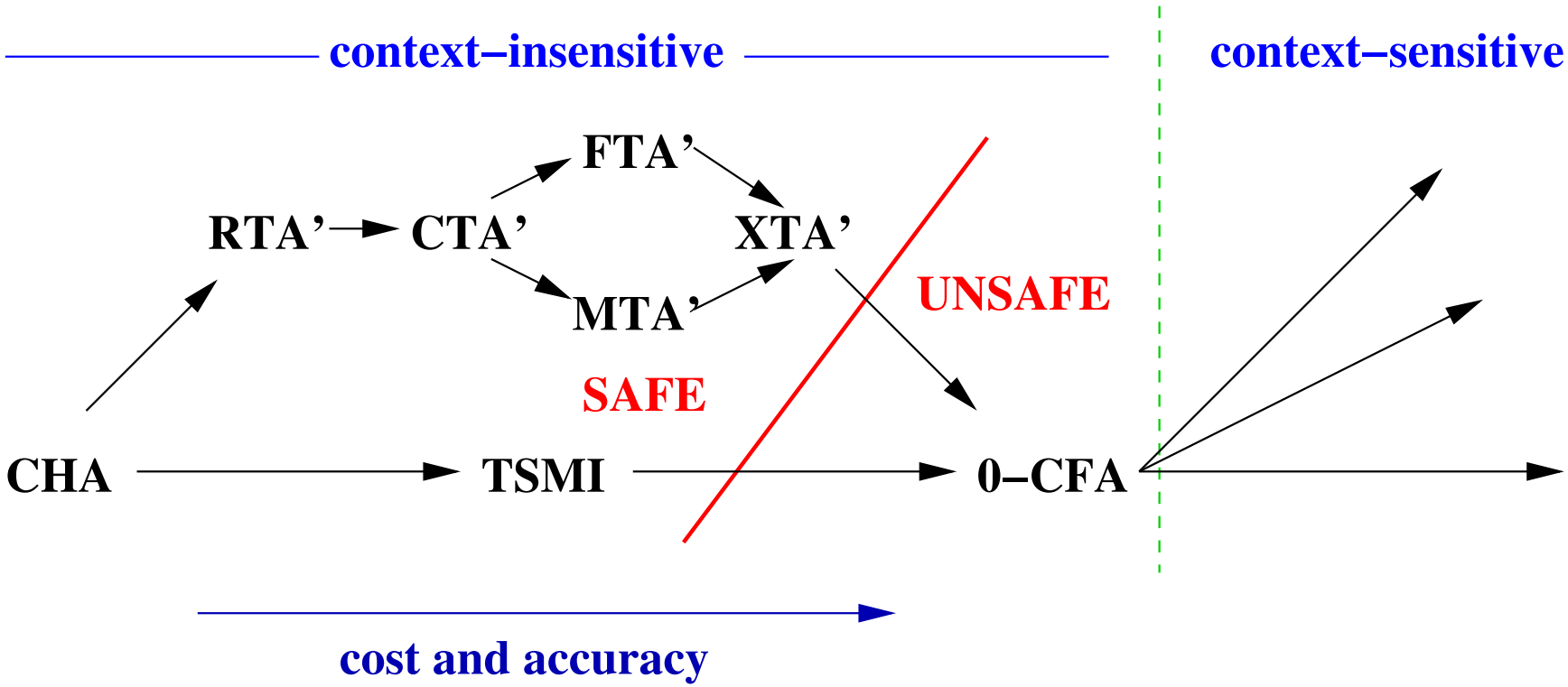


NP-complete to decide whether Java typing is possible.

Conclusion: sometimes one should not inline!

Optimization versus Typability

A more optimistic view:



Inlining does not preserve typability (1/2)

```
B x = new C();
x.m();
```

```
class B {
    void m() {...}
}

class C extends B {
    C f;
    void m() {
        this.f = this;
    }
}
```

Inlining does not preserve typability (2/2)

```
class B {
    void m() {...}
}

class C extends B {
    C f;
    void m() {
        this.f = this;
    }
}
```

```
B x = new C();
x.f = x; // does not type check!
```

Expensive approaches

Type casts: [Wright, Jagannathan et al. TIC'98]

Type casts may hurt performance.

```
class B {
    void m() {...}
}

class C extends B {
    C f;
    void m() {
        this.f = this;
    }
}
```

```
B x = new C();
((C)x).f = (C)x; // two type casts!
```

Limited type inference, else type casts: [Hendren et al. SAS'00]

Type inference for objects:

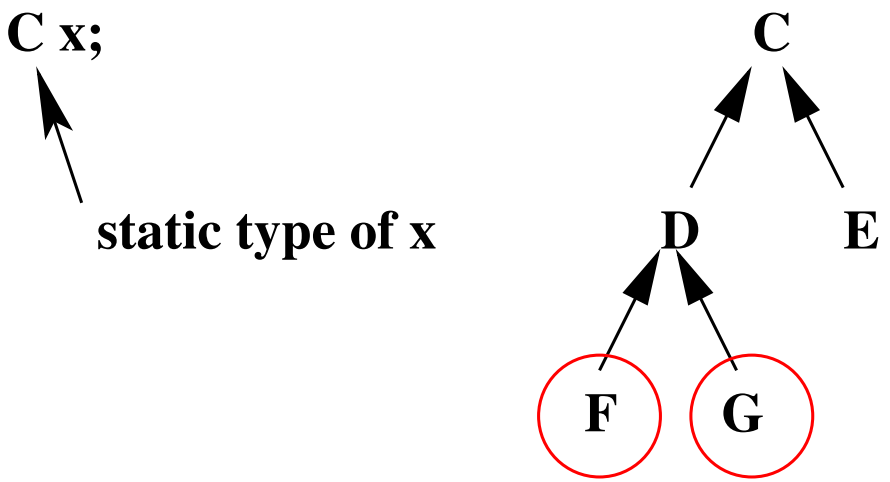
[Palsberg et al. I&C 1995, NJC 1997, FOOL'02, LICS'02]

Our approach: TSMI

Our approach = flow analysis + change types + inline.

Paradigm: type = least-upper-bound(flow-set)

class hierarchy (excerpt)



Flow set for x = { F, G }

New type for x = l.u.b. { F, G } = D

TSMI

Key insight: need to tune the flow analysis to match the type system.

[Palsberg et al. IPL 1992, I&C 1995, TOPLAS 1995 + 1998, JFP 2001]

TSMI is more conservative than 0-CFA.

TSMI imposes more flow constraints than 0-CFA, for example:

- For each class C, the constraint: C $\in$ [this]
- For each method m that overrides another method m, the constraints:
  - the flow sets for corresponding arguments must be equal
  - the flow sets for the return values must be equal
- No flow set can be empty.

Example of TSMI (1/2)

```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

Example of TSMI (2/2)

```
A x = new A();
B y = new B();

x.m( new Q() );
y.m( new S() );
```

```
class A {
    void m(Q arg) {
        arg.p();
    }
}
class B extends A {
    void m(Q arg) {...}
}
```

```
class Q {
    void p() {...}
}
class S extends Q {
    void p() {...}
}
```

TSMI handles the problematic case correctly (1/2)

```
B x = new C();
x.m();
```

```
class B {
    void m() {...}
}

class C extends B {
    C f;
    void m() {
        this.f = this;
    }
}
```

TSMI handles the problematic case correctly (2/2)

```
C x = new C();
x.f = x;
```

```
class B {
    void m() {...}
}

class C extends B {
    C f;
    void m() {
        this.f = this;
    }
}
```

TSMI: Lessons learned

Student experience: **very** difficult to align the flow analysis with the Java type system.

Our experience: we debugged TSMI over and over while doing the correctness proof.

Moral: first theory, then implementation.

Correctness

Based on Featherweight Java [Igarashi, Pierce, Wadler, OOPSLA 1999]

Subset of the Java grammar, small-step operational semantics,
type system, TSMI flow analysis, flow-directed inlining.

ϕ = flow information for the program $\mathtt{P}$ computed by TSMI.

$[\![\mathtt{P}]\!]_\phi$ = program with transformed types, before inlining

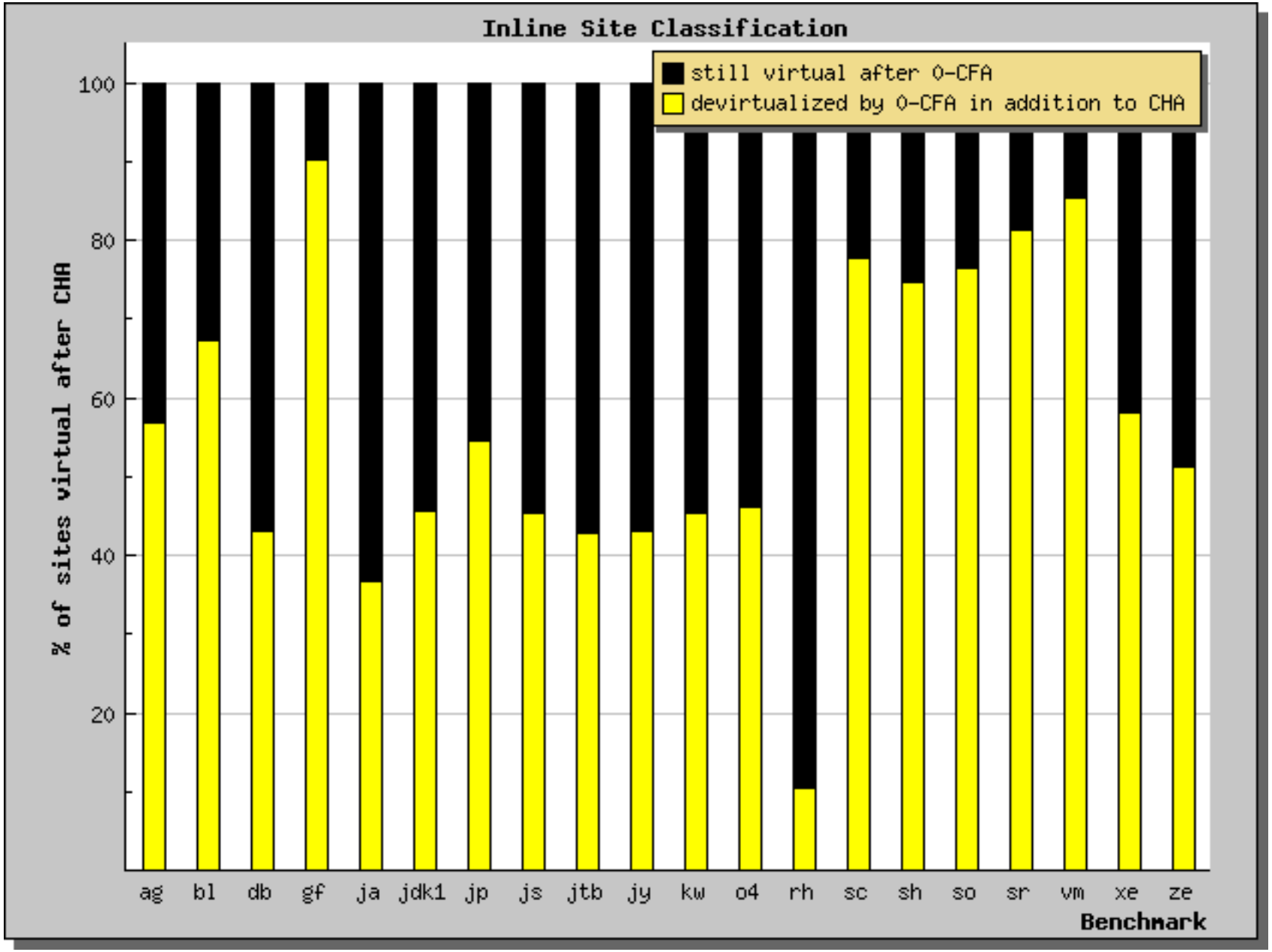
Theorem 1 (Typability Preservation) *If* $\mathtt{P}$ *type checks, then* $[\![\mathtt{P}]\!]_\phi$ *type checks.*

Theorem 2 (Operational Correctness) $\mathtt{P} \mapsto \mathtt{P}'$ *if and only if* $[\![\mathtt{P}]\!]_\phi \mapsto [\![\mathtt{P}']\!]_\phi$.

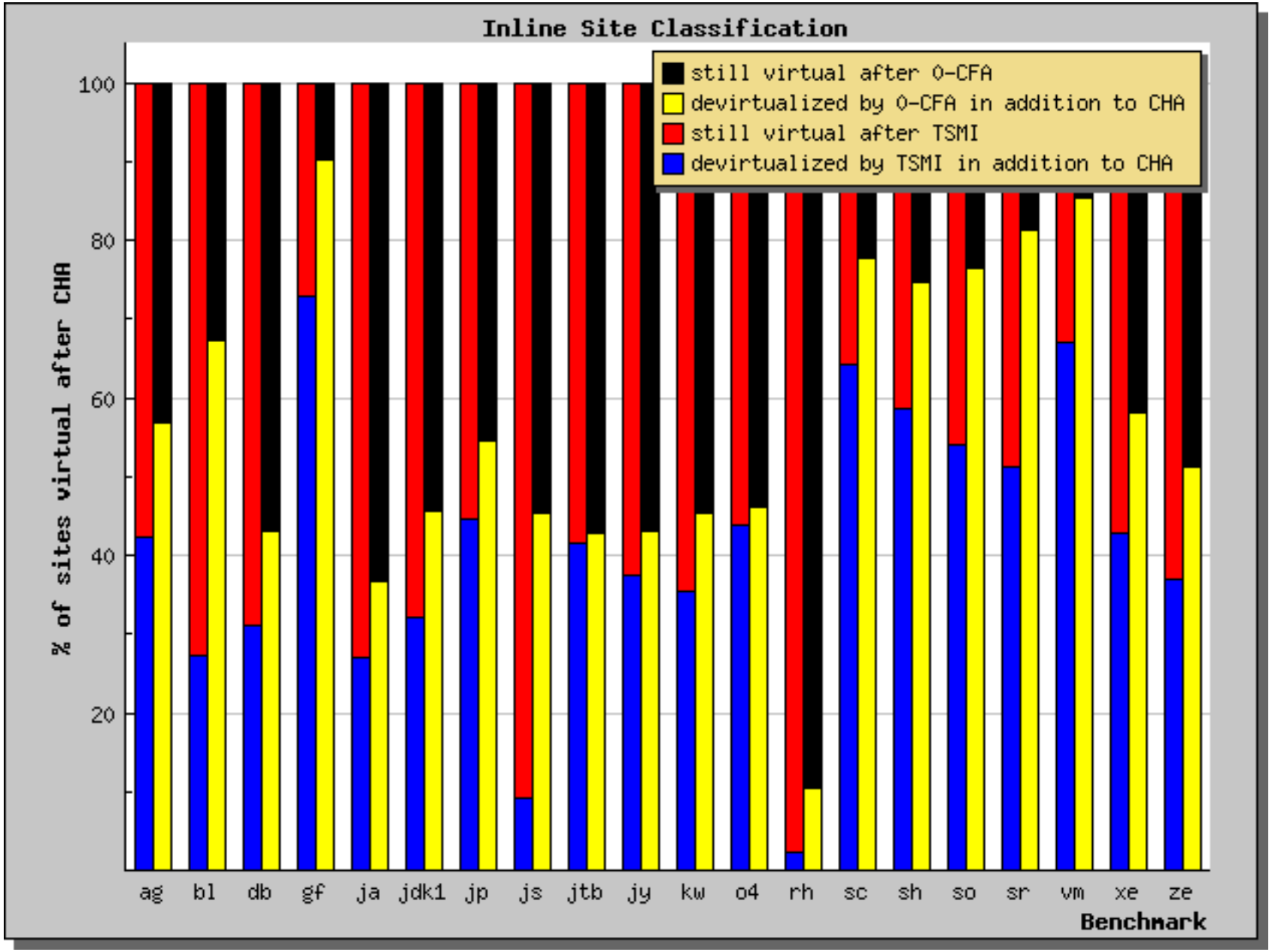
Theorem 3 (Analysis Preservation) ϕ *is a TSMI flow analysis of* $[\![\mathtt{P}]\!]_\phi$.

Theorem 4 (Idempotence) $[\![[\![\mathtt{P}]\!]_\phi]\!]_\phi = [\![\mathtt{P}]\!]_\phi$.

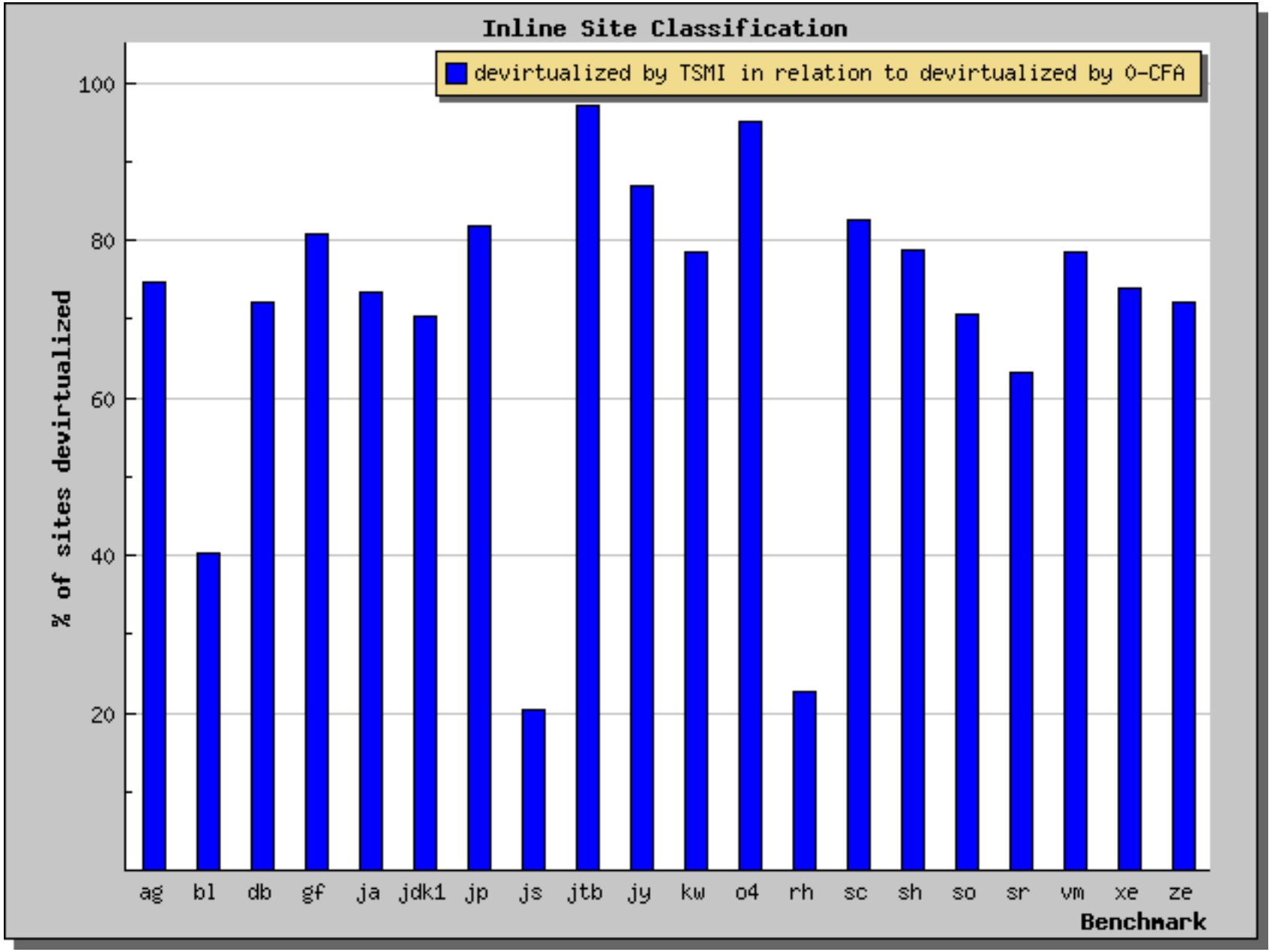
How good is 0-CFA? (revisited)



How good is TSMI?



How good is TSMI relative to 0-CFA?



Conclusion

Type-safe method inlining, based on a new flow analysis.

Almost as good as 0-CFA and potentially much faster and more lightweight.